Chapter 2

Python for Financial Analysis

Python has become one of the most important tools in modern finance. It is flexible, readable, and supported by a rich ecosystem of libraries for data analysis, visualization, statistics, and machine learning. For quantitative researchers, analysts, and practitioners, Python offers a practical way to turn financial data into insight. It is also a gateway into data science more broadly, because many techniques used in finance are the same ones used in business analytics, scientific computing, and artificial intelligence.

This chapter introduces Python from a financial perspective, through common tasks: reading market data, cleaning historical prices, calculating returns, visualizing trends, and preparing data for modeling. Every concept is tied to one running numerical example, a two-asset portfolio over five trading days, so that the reader can check each computation by hand and reproduce it in code. The emphasis is on intuition and practical fluency rather than computer science theory.

The chapter builds up in layers, all on the same AssetA/AssetB price history. It starts with a one-page refresher of the language (functions, conditionals, loops, and the built-in containers) for readers who already program, then introduces the `DataFrame` and NumPy arrays that everything downstream depends on.

It then turns to pandas, the library that does most of the practical work in this book: boolean indexing, merging and grouping, resampling to a coarser frequency, and cleaning a simulated missing observation. A worked example builds and evaluates the two-asset portfolio in full, computing its mean, variance, and covariance by hand and in code, then adds a third asset to show that the logic does not change as a portfolio grows. A further section turns those daily statistics into the numbers a performance report quotes: log returns, annualized volatility, the Sharpe ratio, maximum drawdown, and returns computed from prices adjusted for splits and dividends.

The chapter closes with two practical concerns: why vectorized NumPy code can be tens of times faster than an equivalent Python loop, and how to keep a numerical workflow reproducible so that someone else, or a future self, can rerun it and get the same answer.

Sections 2.1.2 through 2.2.1 establish the vocabulary (loops, arrays, vectorized operations) that later chapters assume. Section 2.4.1's two-asset portfolio recurs by name in Chapters 8 and 12, so its covariance matrix and correlation are worth internalizing. Section 2.4.4 defines the return and performance measures later chapters quote, and Sections 2.3.4 and 2.5.1 anticipate two problems, messy data and slow unvectorized code, that resurface in nearly every applied chapter.

2.1 The Python Language

Everything later in this book is expressed in code, so the language itself has to come first. This part moves from why Python won the quantitative-finance argument to a brief refresher of its syntax and to the `DataFrame` that holds the chapter's running example.

2.1.1 Why Python Matters in Finance

A financial analyst often works with large, messy, and changing datasets. Prices arrive continuously, statements are published irregularly, and news can affect markets within seconds. This environment needs tools that

Wulin.Suo@Queensu.ca

are fast, reproducible, and easy to adapt, which is where Python excels: scripts can be rerun, workflows automated, and models validated systematically.

Python also has a large scientific community. Libraries such as NumPy, pandas, Matplotlib, SciPy, and scikit-learn make numerical work efficient and lower the barrier for researchers moving from spreadsheets to a more scalable and transparent pipeline. Table 2.1 summarizes the libraries used throughout this chapter and the role each one plays.

Library	Role in financial analysis
NumPy	Fast array operations and vectorized numerical computation
pandas	Tabular and time-indexed data handling, cleaning, and aggregation
Matplotlib	Static charting of prices, returns, and diagnostics
SciPy	Statistical distributions, optimization, and hypothesis testing
statsmodels	Regression, time series models (e.g., ARIMA), and statistical inference
scikit-learn	Machine learning models, preprocessing, and model evaluation

Table 2.1: Core Python libraries for financial analysis

2.1.2 A One-Page Python Refresher

This book assumes a reader who already programs, so the language needs only a refresher, built around calculations that recur in finance. A function turns a rule into something reusable, here the future value of \$100 earning 8% a year; an `if/elif/else` chain classifies an outcome; a `for` loop repeats a step. Python's built-in containers (dictionaries for lookups, immutable tuples for fixed pairs, list comprehensions for compact filtering) hold the data before it is tidy enough for a `DataFrame`:

```
def future_value(price, rate, periods=1):
    return price * (1 + rate) ** periods

print(future_value(100.0, 0.08))             # 108.0
print(future_value(100.0, 0.08, periods=3))  # 125.97120000000001

def classify_return(r):
    if r > 0.01:
        return "strong gain"
    elif r > 0:
        return "small gain"
    elif r == 0:
        return "flat"
    else:
        return "loss"

print([classify_return(r) for r in [0.02, 0.004, 0.0, -0.015]])
# ['strong gain', 'small gain', 'flat', 'loss']

balance = 0.0                   # $200 deposited monthly at 0.4% a month
for month in range(12):
    balance = balance * (1 + 0.004) + 200.0
print(round(balance, 2))        # 2453.51

security_info = {"ticker": "AAPL", "sector": "Technology", "price": 190.50}
quote = (100.20, 100.35)        # (bid, ask): a tuple cannot be modified
```

Wulin.Suo@Queensu.ca

```
prices_list = [190.50, 145.20, 102.75]
expensive = [p for p in prices_list if p > 150]
print(security_info["sector"], round(quote[1] - quote[0], 2), expensive)
# Technology 0.15 [190.5]
```

One distinction in this block matters for the rest of the chapter. The deposit loop is sequential, since next month's balance depends on this month's, so a loop (or an equivalent closed-form formula) is the natural way to express it. Most return calculations are the opposite: the same independent operation applied to every observation. For those, the vectorized array operations of Section 2.2.1 are far faster than a loop, as Section 2.5.1 measures. Recognizing which kind of calculation is at hand is one of the most useful judgment calls a financial programmer makes.

2.1.3 Core Data Structures

An analyst tracking even a handful of securities quickly needs more than lists and dictionaries: an ordered run of daily closing prices for several assets, aligned on the same dates. For most financial analysis the key structure is therefore the pandas `DataFrame`, which organizes tabular data in rows and columns.

A `DataFrame` suits financial data because it resembles a spreadsheet: one column may contain dates, another prices, another trading volume. The chapter's running example is a small two-asset price history, built from Python lists so that it is self-contained and reproducible without any external file or internet connection:

```
import numpy as np
import pandas as pd

dates = pd.date_range("2024-01-02", periods=5, freq="B")
prices = pd.DataFrame(
    {
        "AssetA": [100.00, 102.00, 101.00, 104.00, 103.00],
        "AssetB": [50.00, 50.50, 51.26, 51.00, 52.02],
    },
    index=dates,
)
print(prices)
#             AssetA  AssetB
# 2024-01-02  100.00   50.00
# 2024-01-03  102.00   50.50
# 2024-01-04  101.00   51.26
# 2024-01-05  104.00   51.00
# 2024-01-08  103.00   52.02
```

This makes it easy to filter observations, compute statistics, and merge datasets. Here the `DataFrame` holds daily closing prices for two assets, so their performance can be compared and, later in the chapter, combined into a portfolio.

2.2 Numerical Computing with NumPy

Python's built-in containers are general-purpose and, for numerical work, slow. This part introduces the array, the data structure the rest of the book computes with, and the vectorized style of expressing conditions over it rather than looping.

2.2.1 Working with Numerical Arrays

Computing a return one day at a time in a Python loop works for the four prices used so far, but a real price history can span thousands of trading days across hundreds of securities, and a Python-level loop over

Wulin.Suo@Queensu.ca

all of them is far slower than it needs to be. NumPy, the foundational package for numerical computation in Python, provides efficient arrays that support vectorized operations. Vectorization suits finance because many tasks apply the same operation to large collections of prices or returns, with no explicit loop.

Given a series of daily prices $P_0, P_1, \ldots, P_T$, the simple (arithmetic) return in period t is

$$R_t = \frac{P_t - P_{t-1}}{P_{t-1}}. \tag{2.1}$$

Applied to AssetA's five prices, this produces four daily returns:

$$R_1 = \frac{102.00 - 100.00}{100.00} = 2.00\%, \quad R_2 = \frac{101.00 - 102.00}{102.00} = -0.98\%,$$

$$R_3 = \frac{104.00 - 101.00}{101.00} = 2.97\%, \quad R_4 = \frac{103.00 - 104.00}{104.00} = -0.96\%.$$

The mean of these four returns is 0.76%, and their sample standard deviation (with $n - 1 = 3$ in the denominator) is 2.03%. In NumPy, this is computed directly on the underlying array without a Python-level loop:

```
asset_a = prices["AssetA"].to_numpy()
returns_a = (asset_a[1:] - asset_a[:-1]) / asset_a[:-1]
print(np.round(returns_a, 4))
# [ 0.02   -0.0098  0.0297 -0.0096]
print(round(returns_a.mean(), 4), round(returns_a.std(ddof=1), 4))
# 0.0076 0.0203
```

This is faster and more readable than an explicit loop, and arrays match the mathematical structure of quantitative finance, where calculations operate on vectors of observations rather than single numbers.

2.2.2 Boolean Indexing and Vectorized Conditionals

The classification function in Section 2.1.2 applied an `if/elif` chain one return at a time, the kind of loop that scales poorly as a price history grows. NumPy and pandas offer a vectorized alternative. A boolean mask, an array of `True/False` values produced by comparing an array to a condition, selects or counts observations directly:

```
up_days = returns_a > 0
print(up_days)
# [ True False  True False]
print(f"Number of up days: {up_days.sum()}")
# Number of up days: 2
print(f"Average return on up days: {returns_a[up_days].mean():.4f}")
# Average return on up days: 0.0249
```

Summing a boolean array counts the number of `True` values, since Python treats `True` as 1 and `False` as 0, and using the mask to index the array selects only the matching elements. For a conditional calculation with more than two outcomes, `np.select` generalizes the earlier `if/elif` logic into a single vectorized expression:

```
conditions = [returns_a > 0.01, returns_a > 0, returns_a == 0]
choices = ["strong gain", "small gain", "flat"]
labels = np.select(conditions, choices, default="loss")
print(labels)
# ['strong gain' 'loss' 'strong gain' 'loss']
```

This produces the same classifications as the loop-based `classify_return` function, applied to all four returns at once. As datasets grow from a handful of observations to millions of rows of tick data, a single vectorized expression instead of a Python-level loop is the difference between a script that runs in milliseconds and one that takes minutes (Section 2.5.1).

Wulin.Suo@Queensu.ca

2.3 Handling Data with pandas

Financial data arrives as tables that are misaligned, differently sampled, and incomplete. This part covers the library built for that: reshaping and joining tables, plotting what they contain, and cleaning what is wrong with them before any model sees it.

2.3.1 Data Manipulation with pandas

Real market data rarely arrives as clean as the toy price arrays used so far: a downloaded CSV may have gaps for holidays, dates out of order, or days missing for one security but not another, and NumPy knows nothing about dates. pandas is the workhorse library for time series and structured data. It loads data from CSV files, databases, or online sources; indexes by date; sorts observations; handles missing values; and computes rolling statistics, all essential because market data are often irregular and incomplete.

The pandas method `pct_change()` computes the same simple returns shown above, applied automatically to every column:

```
returns = prices.pct_change().dropna()
print(returns.round(4))
#             AssetA  AssetB
# 2024-01-03  0.0200  0.0100
# 2024-01-04 -0.0098  0.0150
# 2024-01-05  0.0297 -0.0051
# 2024-01-08 -0.0096  0.0200
```

Notice that AssetB's returns (0.0100, 0.0150, −0.0051, 0.0200) move in the opposite direction from AssetA's returns (0.0200, −0.0098, 0.0297, −0.0096) on three of the four days. This pattern reappears as a strong negative correlation later in the chapter, and it is why diversification helps so much in this example, although, as Section 2.4.1 shows, four observations pin that correlation down only loosely.

This transformation from price levels to returns underlies much of empirical finance. Returns suit many statistical models better than levels because they are approximately stationary and easier to compare across assets with very different price scales.

2.3.2 Combining and Reshaping Data: merge, groupby, and resample

Real analyses rarely involve a single tidy table; data about the same securities usually arrives from several sources and must be combined. Suppose one table holds each security's sector and price, and a second holds the number of shares held in a portfolio:

```
securities_df = pd.DataFrame([
    {"ticker": "AAPL", "sector": "Technology", "price": 190.50},
    {"ticker": "JPM", "sector": "Financials", "price": 145.20},
    {"ticker": "XOM", "sector": "Energy", "price": 102.75},
    {"ticker": "MSFT", "sector": "Technology", "price": 375.10},
])
shares = pd.DataFrame({
    "ticker": ["AAPL", "JPM", "XOM", "MSFT"],
    "shares_held": [100, 50, 200, 30],
})

merged = securities_df.merge(shares, on="ticker")
merged["position_value"] = merged["price"] * merged["shares_held"]
print(merged)
#   ticker      sector   price  shares_held  position_value
# 0   AAPL  Technology  190.50          100         19050.0
# 1    JPM  Financials  145.20           50          7260.0
```

Wulin.Suo@Queensu.ca

```
# 2    XOM      Energy  102.75          200         20550.0
# 3   MSFT  Technology  375.10           30         11253.0
```

The `merge` method aligns the two tables on the shared `ticker` column, the same logic as a SQL join, and is the standard way to combine data that arrives from different sources or systems. Once combined, `groupby` aggregates the result by a categorical column, here computing total position value by sector:

```
sector_summary = merged.groupby("sector")["position_value"].sum()
print(sector_summary)
# sector
# Energy        20550.0
# Financials     7260.0
# Technology    30303.0
# Name: position_value, dtype: float64
```

This shows at once that the portfolio's Technology exposure ($30,303, combining AAPL and MSFT) is its largest sector concentration, which would take considerably more manual bookkeeping without `groupby`.

A related operation, `resample`, aggregates a time-indexed series to a coarser frequency, for example converting daily prices to weekly prices by taking the last observation in each week:

```
extended_dates = pd.date_range("2024-01-02", periods=10, freq="B")
extended_prices = pd.Series(
    [100.00, 102.00, 101.00, 104.00, 103.00, 105.00, 107.00, 106.00, 108.00, 110.00],
    index=extended_dates, name="AssetA",
)
weekly_last = extended_prices.resample("W-FRI").last()
print(weekly_last)
# 2024-01-05    104.0
# 2024-01-12    108.0
# 2024-01-19    110.0
# Freq: W-FRI, Name: AssetA, dtype: float64

weekly_return = weekly_last.pct_change().dropna()
print(weekly_return.round(4))
# 2024-01-12    0.0385
# 2024-01-19    0.0185
```

Computing weekly returns from resampled weekly prices gives the same answer as compounding the daily simple returns within each week, $\prod_t(1 + r_t) - 1$; for the week ending January 12, both give 0.0385. It is not the same as summing the daily returns, which gives 0.0384 here and is only an approximation for simple returns. Resampling is the more direct and less error-prone method when the analysis needs only the coarser frequency.

2.3.3 Visualization

A table of a hundred daily returns is hard to read at a glance. That two assets tend to move in opposite directions, or that a return series has grown more volatile, is often invisible in a column of numbers but obvious once plotted. Charts reveal patterns in returns, volatility, trends, and correlations, and Matplotlib is the standard library for static plots.

The chapter's `AssetA`/`AssetB` example has only four daily returns, enough to illustrate the portfolio formulas in Section 2.4.1 by hand, but too few for an informative histogram or scatter plot. To show what these charts look like, this section simulates a longer, one-year (252 trading day) two-asset price history with a similar negative correlation between the assets, used only for the three plots below:

```
rng = np.random.default_rng(6)
n_days = 252
```

Wulin.Suo@Queensu.ca

```
mean_returns = [0.0006, 0.0004]
vols = [0.015, 0.010]
corr = -0.6
cov = [[vols[0]**2, corr*vols[0]*vols[1]],
       [corr*vols[0]*vols[1], vols[1]**2]]

daily_returns = rng.multivariate_normal(mean_returns, cov, size=n_days)
dates = pd.bdate_range("2024-01-02", periods=n_days)
returns_viz = pd.DataFrame(daily_returns, index=dates, columns=["AssetA", "AssetB"])
prices_viz = pd.DataFrame({
    "AssetA": 100 * (1 + returns_viz["AssetA"]).cumprod(),
    "AssetB": 50 * (1 + returns_viz["AssetB"]).cumprod(),
})
```

A line chart of the two price series and their cumulative returns shows whether the assets move together or apart. Because AssetA and AssetB start at different price levels ($100 versus $50), the left panel gives each its own y-axis with `twinx()`, so that neither series dwarfs the other on a shared scale:

```
import matplotlib.pyplot as plt

fig, axes = plt.subplots(1, 2, figsize=(10, 4))

ax_a = axes[0]
line_a, = ax_a.plot(prices_viz.index, prices_viz["AssetA"],
                    color="tab:blue", label="AssetA")
ax_a.set_ylabel("AssetA price", color="tab:blue")
ax_a.tick_params(axis="y", labelcolor="tab:blue")
ax_a.set_title("Prices")

ax_b = ax_a.twinx()
line_b, = ax_b.plot(prices_viz.index, prices_viz["AssetB"],
                    color="tab:orange", label="AssetB")
ax_b.set_ylabel("AssetB price", color="tab:orange")
ax_b.tick_params(axis="y", labelcolor="tab:orange")
ax_a.legend(handles=[line_a, line_b], loc="upper left")

(1 + returns_viz).cumprod().plot(ax=axes[1], title="Cumulative growth of $1")
plt.tight_layout()
plt.savefig("ch2_fig_prices_growth.png", dpi=150)
```

Visualization is part of the analysis, not decoration. A model may look strong in a statistical summary while a plot reveals outliers, regime shifts, or data problems that would otherwise go unnoticed. It is often the fastest way to see that two return series move in opposite directions, as AssetA and AssetB do here.

Two other chart types are used constantly in financial analysis. A histogram of returns shows the shape of the return distribution, revealing skewness or heavy tails that summary statistics alone can obscure:

```
fig, ax = plt.subplots(figsize=(6, 4))
returns_viz["AssetA"].hist(ax=ax, bins=20)
ax.set_title("Distribution of AssetA daily returns")
ax.set_xlabel("Return")
plt.tight_layout()
```

Because the simulated returns are drawn from a normal distribution, Figure 2.2 can show neither skewness nor heavy tails, only sampling noise around a bell shape. Real daily returns typically have noticeably heavier tails than a normal distribution, the feature a histogram of real data would expose and the market-risk measures of Chapter 8 must contend with.

Wulin.Suo@Queensu.ca

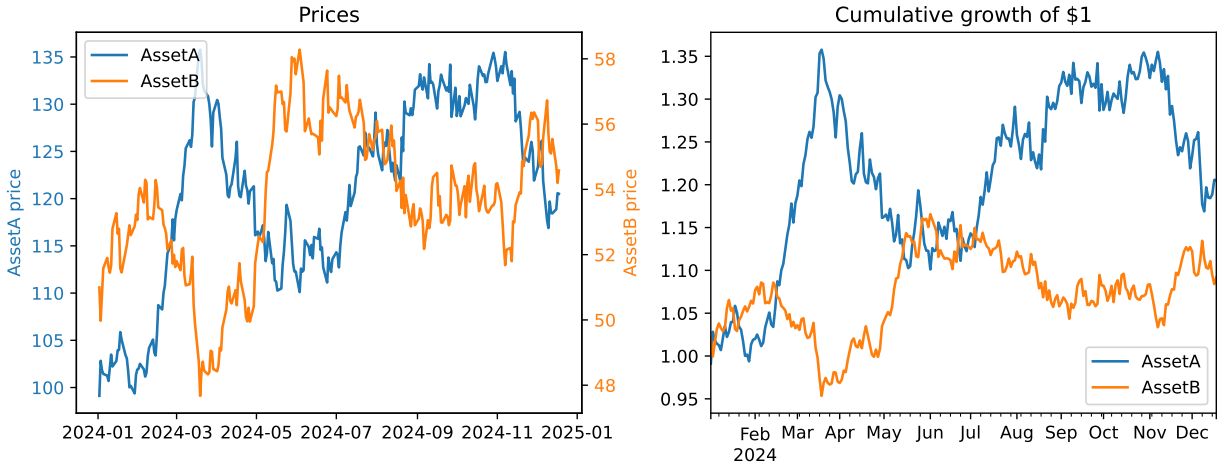


Figure 2.1: Simulated AssetA and AssetB prices, each on its own y-axis (left); cumulative growth of $1 invested in each (right), over a 252-day history

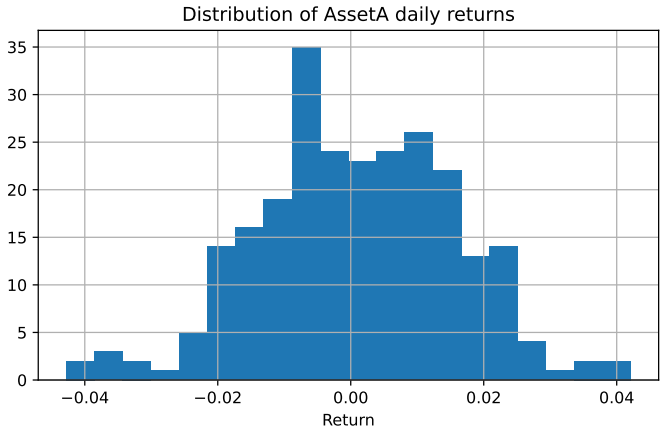


Figure 2.2: Distribution of AssetA's simulated daily returns over the 252-day history

A scatter plot of one asset's returns against another's is the most direct way to visualize a correlation like the one computed numerically in Section 2.4.1:

```
fig, ax = plt.subplots(figsize=(5, 5))
ax.scatter(returns_viz["AssetA"], returns_viz["AssetB"], alpha=0.6)
ax.set_xlabel("AssetA return")
ax.set_ylabel("AssetB return")
ax.set_title(f"Correlation: {returns_viz['AssetA'].corr(returns_viz['AssetB']):.3f}")
plt.tight_layout()
```

Figure 2.3 shows the cloud of 252 points sloping visibly downward, consistent with the simulated correlation of -0.641 printed in the plot title: the two assets tend to move in opposite directions. In a real analysis, the same chart is a useful diagnostic for nonlinear relationships or clusters that a single, linear correlation number cannot capture. The chapter's four-observation example (Section 2.4.1) shows the same pattern with an even stronger correlation of -0.896, but has too few points to plot informatively.

Wulin.Suo@Queensu.ca

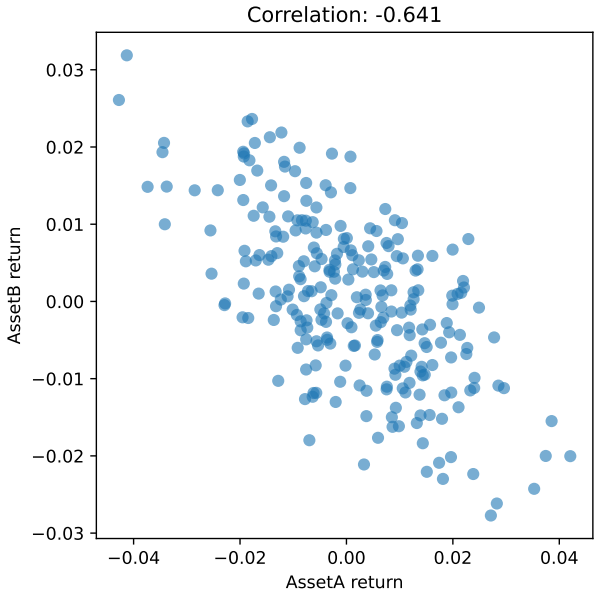


Figure 2.3: Simulated AssetA daily returns plotted against simulated AssetB daily returns

2.3.4 Data Cleaning and Preparation

Raw financial data often contains missing entries, duplicate observations, outliers, inconsistent dates, or formatting errors, so data cleaning is a major part of practical work. A well-designed pipeline may align multiple time series, fill missing observations carefully, adjust for corporate actions such as dividends and splits, and remove clearly erroneous values.

Figure 2.4 shows a simple financial data workflow, and the code below shows a typical cleaning step: forward-filling a small gap in a price series (for example, caused by a data feed outage) rather than silently dropping the observation, which would misalign the two return series.

```
raw = prices.copy()
raw.loc["2024-01-04", "AssetB"] = np.nan  # simulate a missing quote
cleaned = raw.ffill()
print(raw["AssetB"].isna().sum(), "missing value(s) before cleaning")
print(cleaned["AssetB"].isna().sum(), "missing value(s) after cleaning")
# 1 missing value(s) before cleaning
# 0 missing value(s) after cleaning
```



Figure 2.4: A typical financial data pipeline moves from raw data to cleaned inputs and then to modeling

Forward-filling suits a short gap in an otherwise liquid series, but forward-filling an illiquid security for many consecutive days can create the false impression of zero volatility. A stock's price series must also often be adjusted for splits and dividends to produce a consistent total return series; otherwise mechanical price drops that are not economic losses distort its history, as Section 2.4.4 shows on the running example. Even a sophisticated model is only as good as the data that feed it.

Wulin.Suo@Queensu.ca

2.4 Portfolio Analysis in Python

With the tools in place, this part puts them to work on the problem the rest of the book returns to repeatedly: constructing a portfolio from return data, evaluating it, and scaling the same code from two assets to many.

2.4.1 Worked Example 2.1: Building and Evaluating a Two-Asset Portfolio

This section ties the running example together into a small but complete analysis, the kind of workflow an analyst repeats constantly at larger scale. Suppose an analyst wants to evaluate an equally weighted portfolio of AssetA and AssetB.

```
weights = np.array([0.5, 0.5])
portfolio_returns = returns.dot(weights)
print(portfolio_returns.round(4))
# 2024-01-03    0.0150
# 2024-01-04    0.0026
# 2024-01-05    0.0123
# 2024-01-08    0.0052
# dtype: float64

port_mean = portfolio_returns.mean()
port_vol = portfolio_returns.std(ddof=1)
print(round(port_mean, 4), round(port_vol, 4))
# 0.0088 0.0058
```

Compare the portfolio's volatility of 0.58% to the weighted average of the two assets' individual volatilities, $0.5 \times 2.03\% + 0.5 \times 1.08\% = 1.56\%$. The portfolio is far less volatile than either asset alone, and than a naive average of their volatilities would suggest. This is the diversification effect introduced in Chapter 1, now made precise. The covariance between the two return series is

```
cov_matrix = returns.cov()
print(cov_matrix.round(6))
#          AssetA    AssetB
# AssetA  0.000414 -0.000198
# AssetB -0.000198  0.000118

corr = returns.corr().iloc[0, 1]
print(round(corr, 3))
# -0.896
```

so the two assets have a strong negative correlation of about -0.90. That estimate rests on only four pairs of returns, however, and is very imprecise: the standard Fisher-z 95% confidence interval runs from about -0.998 to $+0.467$, wide enough to include zero and even a moderately positive correlation. The example is chosen so that every number can be checked by hand, not as evidence about any real pair of assets; Chapter 12 returns to the damage this kind of estimation error does to portfolio optimization. The variance of an equally weighted two-asset portfolio is given by

$$\sigma_p^2 = w_A^2 \sigma_A^2 + w_B^2 \sigma_B^2 + 2 w_A w_B \sigma_{AB}, \tag{2.2}$$

and substituting $w_A = w_B = 0.5$, $\sigma_A^2 = 0.000414$, $\sigma_B^2 = 0.000118$, and $\sigma_{AB} = -0.000198$ gives

$$\sigma_p^2 = 0.25(0.000414) + 0.25(0.000118) + 2(0.25)(-0.000198) = 0.000034,$$

so $\sigma_p = \sqrt{0.000034} \approx 0.58\%$, matching the value computed directly from the portfolio return series. Agreement between the formula and the code is a useful checkpoint: whenever a hand-derived formula and a direct computation disagree, there is a bug in one of them, and reconciling the two is one of the most valuable debugging habits a quantitative analyst can build.

Wulin.Suo@Queensu.ca

Running such analyses quickly, and extending them to many more assets and longer histories, is one reason Python is widely used in finance: researchers can iterate, test ideas, and communicate results reproducibly, which matters where decisions are made under time pressure.

2.4.2 Scaling Up: From Two Assets to a Small Portfolio Universe

The two-asset example was kept small so that every number could be checked by hand, but the code scales to any number of assets without modification. Adding a third asset, AssetC, requires only a new column in the original DataFrame:

```
prices["AssetC"] = [200.00, 198.00, 202.00, 199.00, 203.00]
returns3 = prices.pct_change().dropna()
print(returns3.round(4))
#             AssetA  AssetB  AssetC
# 2024-01-03  0.0200  0.0100 -0.0100
# 2024-01-04 -0.0098  0.0150  0.0202
# 2024-01-05  0.0297 -0.0051 -0.0149
# 2024-01-08 -0.0096  0.0200  0.0201
```

No line of the return, covariance, or portfolio-variance calculation changes; `pct_change()`, `.cov()`, and matrix multiplication work the same way however many columns the DataFrame has:

```
weights3 = np.array([1/3, 1/3, 1/3])
port3 = returns3.dot(weights3)
print(port3.round(4))
# 2024-01-03    0.0067
# 2024-01-04    0.0085
# 2024-01-05    0.0033
# 2024-01-08    0.0102

cov3 = returns3.cov()
port_var3 = weights3 @ cov3.values @ weights3
print(f"Equal-weighted 3-asset portfolio volatility: {np.sqrt(port_var3):.4f}")
# Equal-weighted 3-asset portfolio volatility: 0.0030
```

The expression `weights3 @ cov3.values @ weights3` is the matrix form of the portfolio variance formula, $w^\top \Sigma w$, which Section 2.5.3 restates for n assets, and it works identically whether w has 2, 3, or 200 entries. This is the payoff of writing analysis code in vectors and matrices rather than a named variable per asset: a script built for a two-asset illustration is, without rewriting, the script a researcher would run on hundreds of securities, the setting Chapter 12's optimization methods are built for.

Object-Oriented Programming: A Reusable `Portfolio` Class

The procedural style used so far, a sequence of variable assignments and function calls, works for a one-off analysis. Repeating the same mean, volatility, and variance calculations for every new portfolio, however, invites the copy-paste duplication that this chapter's reproducibility section warns against. Object-oriented programming addresses this with a `class`: a template, defined once, that bundles related data (here, a portfolio's returns and weights) with the functions that operate on it, called *methods*. Every new portfolio is then built from the same reusable blueprint rather than by copying and re-editing calculations:

```
class Portfolio:
    def __init__(self, returns, weights):
        self.returns = returns
        self.weights = weights

    @property
```

Wulin.Suo@Queensu.ca

```
    def portfolio_returns(self):
        return self.returns.dot(self.weights)

    @property
    def mean(self):
        return self.portfolio_returns.mean()

    @property
    def volatility(self):
        return self.portfolio_returns.std(ddof=1)

    def variance_from_formula(self):
        cov = self.returns.cov().values
        return self.weights @ cov @ self.weights
```

Instantiating this class on the two-asset data reproduces every number already computed by hand,

```
portfolio = Portfolio(returns, weights)
print(round(portfolio.mean, 4), round(portfolio.volatility, 4))
# 0.0088 0.0058
print(round(portfolio.variance_from_formula(), 6))
# 3.4e-05
```

but the payoff appears with the three-asset portfolio: `Portfolio(returns3, weights3)` produces the correct mean, volatility, and formula-based variance from the same class definition, with no new code. This is what "scaling up" means in an object-oriented codebase: the `Portfolio` class is a single, tested abstraction that every future portfolio, of any size, can reuse, the kind of reusable component a production research or trading system is built from.

2.4.3 Working with Time Series and Financial Data

An analyst comparing two securities that trade on different exchanges, or lining up daily prices with a monthly economic release, meets a problem that rows and columns alone do not solve: which date does each observation belong to, and what happens when one series has an entry for a day the other lacks? Many financial datasets are time series, with observations indexed by time and ordered sequentially: stock prices, interest rates, exchange rates, and macroeconomic indicators. Such data require careful handling of date indexes, alignment, and missing observations. pandas provides a time-indexed data structure that makes these operations natural, including resampling to a different frequency and computing rolling statistics:

```
rolling_vol = returns["AssetA"].rolling(window=2).std()
print(rolling_vol.round(4))
# 2024-01-03       NaN
# 2024-01-04    0.0211
# 2024-01-05    0.0279
# 2024-01-08    0.0278
# dtype: float64
```

A common workflow imports historical prices, converts them into a time series, computes returns, and then studies autocorrelation, volatility clustering, and trends. Many financial models depend on the future not being independent of the past, yet that dependence is often subtle and nonstationary, which is why Chapter 7 is needed.

2.4.4 Returns, Risk, and Performance in pandas

Every statistic computed so far is a daily figure on simple returns from unadjusted prices. A performance report quotes something else: returns aggregated over time, risk and return stated per year, return per unit of risk, and the worst loss along the way, all computed from prices corrected for splits and dividends. This section defines each of these on the running example; later chapters use them without re-deriving them.

Wulin.Suo@Queensu.ca

Simple and Log Returns

The log (continuously compounded) return over period t is

$$r_t = \ln \frac{P_t}{P_{t-1}} = \ln(1 + R_t), \tag{2.3}$$

where R_t is the simple return of Equation (2.1). The two are close for small moves, $r_t \approx R_t - R_t^2/2$, but they aggregate differently. Log returns add over time, because $\sum_t r_t = \ln(P_T/P_0)$, while simple returns compound: $\prod_t (1+R_t) - 1 = P_T/P_0 - 1$. Simple returns, on the other hand, add across assets: a portfolio's simple return is exactly the weighted average $\sum_i w_i R_{i,t}$ of its holdings' simple returns, and no such identity holds for log returns.

```
log_returns = np.log1p(returns)          # r_t = ln(1 + R_t)
print(log_returns.round(6))
print(round(log_returns["AssetA"].sum(), 6), round(np.log(103 / 100), 6))
print(round(returns["AssetA"].sum(), 6),
      round((1 + returns["AssetA"]).prod() - 1, 6))

day1 = returns.iloc[0]                   # weights = [0.5, 0.5]
print(round(day1 @ weights, 6), round(np.log1p(day1) @ weights, 6),
      round(np.log1p(day1 @ weights), 6))
#                AssetA    AssetB
# 2024-01-03  0.019803  0.009950
# 2024-01-04 -0.009852  0.014937
# 2024-01-05  0.029270 -0.005085
# 2024-01-08 -0.009662  0.019803
# 0.029559 0.029559
# 0.030284 0.03
# 0.015 0.014876 0.014889
```

AssetA's four log returns sum to exactly $\ln(103/100) = 0.029559$, while its four simple returns sum to 0.030284, overstating the true 3.00% gain that compounding them recovers. On the first day the portfolio's simple return, 1.50%, is exactly the average of the two assets' simple returns; the average of their log returns, 0.014876, misses the portfolio's own log return, 0.014889. Hence the working rule this book follows: log returns for aggregating one series through time and for the statistical models of Chapter 7, simple returns whenever assets are combined into a portfolio.

Annualization and the Square-Root-of-Time Rule

Returns and volatilities are conventionally quoted per year, using about 252 trading days. If daily returns have mean μ_d and standard deviation σ_d, then

$$\mu_{\text{ann}} = 252\,\mu_d, \qquad \sigma_{\text{ann}} = \sqrt{252}\,\sigma_d. \tag{2.4}$$

The mean scales with time because expected returns add; the standard deviation scales with the square root of time because the variance of a sum of 252 *independent* daily returns with a common variance is $252\sigma_d^2$. Both scalings lean on assumptions. The rule is exact for log returns that are independent and identically distributed; it is wrong to the extent that daily returns are autocorrelated (positive autocorrelation makes $\sqrt{252}\,\sigma_d$ understate annual risk, negative autocorrelation overstate it) or that volatility itself clusters in calm and turbulent spells, the subject of Chapter 7's GARCH models. It is a convention for comparing numbers on a common scale, not a forecast of next year's range.

Wulin.Suo@Queensu.ca

The Sharpe Ratio

Return alone says nothing about the risk taken to earn it. The Sharpe ratio divides the average excess return over the risk-free rate r_f by the volatility of returns,

$$\mathrm{SR} = \frac{\bar{R} - r_f}{\sigma}, \tag{2.5}$$

with every quantity measured at the same frequency. A daily Sharpe ratio is annualized by multiplying by $\sqrt{252}$, which follows from Equation (2.4) and inherits its independence assumption. For orientation, the whole US stock market has earned an annualized Sharpe ratio of roughly 0.4 over the past century, and a strategy that sustains 1 after costs is considered good. Chapter 12 builds portfolio choice around this ratio, and Chapter 13 shows how easily a search over many strategies manufactures a high one by luck.

Maximum Drawdown

Volatility treats a gain and a loss of the same size alike, and it ignores the order in which they occur. An investor experiences the path: how far the account falls from its previous high before it recovers. If V_t is the value of \$1 invested at the start, the maximum drawdown is the largest such fall,

$$\mathrm{MDD} = \max_{t} \frac{\max_{s \le t} V_s - V_t}{\max_{s \le t} V_s}, \tag{2.6}$$

a path-dependent measure that Chapter 8 develops alongside Value at Risk.

Worked Example 2.2: Performance of AssetA, AssetB, and the Portfolio

The code below puts the three measures together for the two assets and the equally weighted portfolio of Section 2.4.1, with the risk-free rate set to zero for this five-day illustration, as Chapter 12 also does:

```
TRADING_DAYS = 252
series = pd.concat([returns, portfolio_returns.rename("Portfolio")], axis=1)
perf = pd.DataFrame({"mean_d": series.mean(), "vol_d": series.std(ddof=1)})
perf["mean_a"] = perf["mean_d"] * TRADING_DAYS
perf["vol_a"] = perf["vol_d"] * np.sqrt(TRADING_DAYS)
perf["SR_d"] = perf["mean_d"] / perf["vol_d"]        # risk-free rate set to 0
perf["SR_a"] = perf["SR_d"] * np.sqrt(TRADING_DAYS)
print(perf.round(4))
print(round(np.sqrt(TRADING_DAYS / len(series)), 2))  # rough std. error of SR_a
#            mean_d   vol_d  mean_a   vol_a    SR_d     SR_a
# AssetA     0.0076  0.0203  1.9079  0.3229  0.3722   5.9078
# AssetB     0.0100  0.0108  2.5186  0.1721  0.9218  14.6329
# Portfolio  0.0088  0.0058  2.2132  0.0925  1.5064  23.9138
# 7.94

def max_drawdown(r):
    wealth = np.concatenate([[1.0], (1 + r).cumprod().to_numpy()])
    running_peak = np.maximum.accumulate(wealth)
    return ((running_peak - wealth) / running_peak).max()

for name in series:
    print(f"{name:9s} maximum drawdown: {max_drawdown(series[name]):.2%}")
# AssetA    maximum drawdown: 0.98%
# AssetB    maximum drawdown: 0.51%
# Portfolio maximum drawdown: 0.00%
```

Wulin.Suo@Queensu.ca

Every entry can be checked by hand. AssetA's daily volatility of 2.03% annualizes to $2.03\% \times \sqrt{252} = 32.3\%$, a plausible figure for a single stock. Its daily mean of 0.76% annualizes to 191%, and the portfolio's daily Sharpe ratio of $0.88\%/0.58\% = 1.51$, the figure Chapter 12 quotes, annualizes to $1.51 \times \sqrt{252} = 23.9$. Those last two numbers are arithmetically correct and economically meaningless. Four observations cannot pin down a mean, and the standard error of an annualized Sharpe ratio is roughly $1/\sqrt{\text{years of data}}$, here $\sqrt{252/4} = 7.94$, so even 23.9 is barely three standard errors from zero; Chapter 13 derives this standard error and shows how badly it bites on far longer samples. The drawdowns tell the same diversification story as the volatilities: AssetA's worst fall is from \$102 to \$101, 0.98%; AssetB's is from \$51.26 to \$51.00, 0.51%; and the portfolio, which rose on all four days, never fell below a previous high at all.

Adjusted versus Raw Prices

A raw closing price series records what a share traded for, which changes for reasons that are not gains or losses to the holder. In a 2-for-1 stock split each share becomes two, and the price halves overnight; on an ex-dividend date the price drops by roughly the dividend, which the holder receives in cash. Returns computed from raw prices count both as losses. Data vendors therefore publish an adjusted price series in which every price before such an event is multiplied by a factor: the split ratio (0.5 for a 2-for-1 split), and $1 - D/P$ for a dividend D paid after a close of P. Yahoo Finance, for example, applies these split and dividend multipliers following the standards of the Center for Research in Security Prices (CRSP). The code below applies both adjustments to AssetA, first imagining that its last two prices had been reported after a 2-for-1 split, then that the \$1 fall on January 8 was the stock going ex-dividend on a \$1.00 dividend:

```
raw_a = pd.Series([100.00, 102.00, 101.00, 52.00, 51.50], index=prices.index)
split_factor = pd.Series([0.5, 0.5, 0.5, 1.0, 1.0], index=prices.index)
adj_a = raw_a * split_factor             # 2-for-1 split before the Jan 5 open
print(pd.DataFrame({"raw": raw_a.pct_change(),
                    "adjusted": adj_a.pct_change()}).dropna().round(4))

dividend = 1.00                          # AssetA goes ex-dividend on Jan 8
div_factor = 1 - dividend / 104.00       # applied to every close before then
adj_close = prices["AssetA"] * np.where(prices.index < "2024-01-08",
                                        div_factor, 1.0)
print(round(div_factor, 6), adj_close.round(4).tolist())
print(round(103.00 / 104.00 - 1, 4),
      round(adj_close.iloc[-1] / adj_close.iloc[-2] - 1, 4))
#                raw  adjusted
# 2024-01-03  0.0200    0.0200
# 2024-01-04 -0.0098   -0.0098
# 2024-01-05 -0.4851    0.0297
# 2024-01-08 -0.0096   -0.0096
# 0.990385 [99.0385, 101.0192, 100.0288, 103.0, 103.0]
# -0.0096 0.0
```

The raw series shows a 48.5% crash on January 5 that never happened to anyone's wealth; the adjusted series recovers AssetA's true 2.97% gain. In the dividend case the raw price return on January 8 is -0.96%, but the holder's total return, $(103 + 1)/104 - 1$, is zero, which the adjusted series shows. Because the factor multiplies all earlier prices by the same constant, every earlier return is unchanged. Only the level of the old prices moves, and it moves again with each new dividend, which is why an adjusted price downloaded today will not match one downloaded last year even though the returns agree. Use adjusted prices for returns and performance measures, and raw prices for anything tied to the actual traded price, such as an order's limit price or an option's strike. The `yfinance` library used in this chapter's last exercise returns split- and dividend-adjusted prices in its `Close` column by default (`auto_adjust=True`); pass `auto_adjust=False` to see the raw close alongside the adjusted one.

Wulin.Suo@Queensu.ca

2.5 Code That Holds Up

Code that produces the right number once is not the same as code that can be trusted, rerun, and handed to somebody else. This part covers performance, the errors financial data reliably provokes, the summary statistics worth computing, and the practices that make a result reproducible.

2.5.1 Why Vectorization Matters: A Performance Comparison

Section 2.2.1 claimed that vectorized array operations are faster than an equivalent Python-level loop. To demonstrate it, consider computing the cumulative growth of $1 across 100,000 simulated daily returns, once with an explicit loop and once with NumPy's vectorized `cumprod`:

```
import time

rng = np.random.default_rng(0)
simulated_returns = rng.normal(0.0005, 0.01, 100_000)

start = time.perf_counter()
total = 1.0
loop_values = []
for r in simulated_returns:
    total *= (1 + r)
    loop_values.append(total)
loop_time = time.perf_counter() - start

start = time.perf_counter()
vectorized_values = np.cumprod(1 + simulated_returns)
vector_time = time.perf_counter() - start

print(f"Loop: {loop_time:.4f}s, Vectorized: {vector_time:.4f}s, "
      f"speedup: {loop_time/vector_time:.0f}x")
# Loop: 0.0394s, Vectorized: 0.0010s, speedup: 39x
```

Both approaches produce identical final values, but the vectorized version is roughly 39 times faster on this test (the figure is machine-dependent, but a factor of 10 to 100 is typical for this kind of calculation). The Python-level loop pays interpreter overhead (argument checking, dynamic type dispatch) on each of the 100,000 iterations, while `cumprod` executes the equivalent loop once, in compiled C code, over contiguous memory. The gap widens as the dataset grows, which is why Section 2.6 treats vectorized NumPy and pandas operations as the default, reserving explicit loops for path-dependent calculations, like the sequential compounding example in Section 2.1.2, that cannot be vectorized.

2.5.2 Handling Errors and Common Pitfalls

Financial data pipelines fail constantly in small, mundane ways: a data feed returns a missing value, a ticker is misspelled, a division produces infinity because a denominator was zero. Python's `try/except` block lets a program catch such errors and respond deliberately rather than crash:

```
def safe_return(price_now, price_before):
    try:
        return (price_now - price_before) / price_before
    except ZeroDivisionError:
        return float("nan")

print(safe_return(105.0, 100.0))
# 0.05
print(safe_return(105.0, 0.0))
```

Wulin.Suo@Queensu.ca

```
# nan
```

Returning `nan` (not-a-number) rather than crashing lets downstream code, and pandas in particular, continue processing the rest of a dataset while marking the problematic observation, like the missing-value handling in Section 2.3.4. A few pitfalls recur often in financial Python code:

- **Silent `NaN` propagation.** Almost any arithmetic operation involving `NaN` produces another `NaN` rather than an error, so a single bad data point can spread through an entire calculation (a covariance matrix, for example) without warning.
- **Comparing floating-point numbers for exact equality.** Because of how computers represent decimal numbers, a calculation that should equal `0.3` may instead equal `0.30000000000000004`; use a tolerance-based comparison (`np.isclose`) rather than `==` when checking floating-point results.
- **Mutating a DataFrame while iterating over it.** Modifying rows of a DataFrame inside a loop over that same DataFrame can produce confusing, order-dependent results; it is almost always better to build a new column or DataFrame with a vectorized expression.
- **Off-by-one errors in lagged calculations.** Return, rolling-window, and lag calculations are prone to off-by-one mistakes, such as using today's price where yesterday's was intended, which can silently introduce look-ahead bias into a backtest (Section 2.6).

2.5.3 Descriptive Statistics and Portfolio Analytics

A return series that has been imported, cleaned, and plotted still leaves an obvious question: how risky does this asset look, in numbers precise enough to compare against another asset or feed into a portfolio calculation? Descriptive statistics such as averages, standard deviations, skewness, and kurtosis summarize the distribution of returns and are the starting point for risk analysis:

```
summary = returns.describe().round(4)
print(summary)
#        AssetA  AssetB
# count  4.0000  4.0000
# mean   0.0076  0.0100
# std    0.0203  0.0108
# min   -0.0098 -0.0051
# 25%   -0.0097  0.0062
# 50%    0.0052  0.0125
# 75%    0.0224  0.0163
# max    0.0297  0.0200
```

For a portfolio, one also computes correlations, covariance matrices, and portfolio volatility, as in the worked example above. These quantities are central to diversification and asset allocation, and they generalize directly to many assets: with n assets, weights $w \in \mathbb{R}^n$, and covariance matrix Σ, portfolio variance is $\sigma_p^2 = w^\top \Sigma w$, the two-asset formula in matrix form.

The same code applies unchanged whether there are 2 assets or 200. A single script can compute rolling volatilities and the annualized volatilities, Sharpe ratios, and drawdowns of Section 2.4.4 across a whole universe of securities, which matters when a researcher compares strategies, evaluates risk, or tests whether a signal survives transaction costs.

2.5.4 Reproducibility, Version Control, and Good Practice

Analysis in computational finance must be reproducible. A result that cannot be rerun by another analyst, or by the same analyst six months later, is of limited value. Python supports reproducibility through scripts, notebooks, and package management. A clear workflow includes:

Wulin.Suo@Queensu.ca

- Recording exact package versions, for example in a `requirements.txt` or `environment.yml` file, so that a colleague (or a future you) can recreate the same environment.
- Saving raw data separately from cleaned or derived data, so that a cleaning step can be re-run or audited without re-fetching from the original source.
- Documenting transformations, ideally in code and comments rather than in a separate document that can drift out of sync with the script.
- Separating analysis from presentation, so that a notebook used to explore an idea is not the same artifact used to report a result to a manager or regulator.
- Using version control (such as Git) to track changes to code over time, including which version of the code produced a given historical result.
- Fixing random seeds (as in `np.random.default_rng(0)` in Section 2.5.1) whenever a calculation involves simulation, so that two runs of the same script on the same data produce identical results.
- Writing small checks into a script or notebook, such as the put-call parity check of Chapter 5 or the accounting-identity check of Chapter 4, so that a silent bug is caught immediately rather than after it has propagated into a report or a trading decision.

2.5.5 Unit Testing: Making Checks Automatic

The "small checks" above become far more valuable once written as automatically run tests rather than one-off `print` statements an analyst must remember to inspect. A unit test is a function that exercises a piece of code and uses Python's `assert` statement to fail loudly if the result is not what is expected. Section 2.4.1's observation that the portfolio-variance formula and the direct computation agree is the kind of invariant worth encoding this way:

```
def test_portfolio_variance_matches_formula():
    weights = np.array([0.5, 0.5])
    cov = returns.cov().values
    var_A, var_B, cov_AB = cov[0, 0], cov[1, 1], cov[0, 1]
    formula_variance = (
        weights[0]**2 * var_A + weights[1]**2 * var_B
        + 2 * weights[0] * weights[1] * cov_AB
    )
    direct_variance = returns.dot(weights).var(ddof=1)
    assert abs(formula_variance - direct_variance) < 1e-10, \
        "Portfolio variance formula does not match direct computation!"
    print("Test passed: formula-based and direct portfolio variance agree.")

test_portfolio_variance_matches_formula()
# Test passed: formula-based and direct portfolio variance agree.
```

The tolerance, 10^{-10}, is loose enough to absorb ordinary floating-point rounding but tight enough to catch a real error, such as a transposed covariance term or a forgotten factor of two on the cross term. Python's own `statistics.mean` shows how quietly such bugs arise: handed a list of NumPy integers such as prices of 100, 101, and 103, it returns the truncated integer 101 rather than 101.3333, with no error or warning, a wrong answer that only a check against an independently computed value, here `np.mean`, would catch. A test like this costs a few lines and runs in a fraction of a second, but it turns a fact verified once by hand into one re-verified every time the surrounding code changes, catching a regression immediately rather than when a downstream number looks suspicious. Production financial codebases collect dozens or hundreds of such tests, run automatically before any code change is accepted with a framework such as `pytest`, but the principle is the same at any scale: encode a known-correct result as an executable check rather than a comment.

Wulin.Suo@Queensu.ca

Reproducibility is not a bureaucratic nicety; in finance it is often a legal and professional requirement. A regulator investigating a trading loss, an auditor reviewing a valuation, or a portfolio manager questioning a backtest will expect to trace a reported number back to the code and data that produced it, possibly months or years after the fact. A workflow that cannot support this scrutiny is a liability however sophisticated its models.

Good practice also means writing code that others can understand. Financial projects often involve several collaborators, long time horizons, and changing requirements, and a readable function, a clear variable name, and a documented transformation prevent errors and make the analysis easier to audit.

2.6 Challenges in Applying Python to Financial Data

Several practical challenges recur when Python is used for financial analysis.

Data quality and vendor discrepancies. Different data vendors can report slightly different prices, adjust for corporate actions differently, or use different conventions for holidays and time zones. A workflow that silently trusts a single source can propagate vendor-specific errors into every downstream result.

Look-ahead and survivorship bias. It is easy to use information that would not have been available at the time of a decision, for example by studying past performance with a company's current list of index constituents, which silently excludes firms that were delisted or went bankrupt. Both biases can make a backtest look far more profitable than any strategy could have been in real time.

Non-stationarity and changing data schemas. Return distributions, volatility regimes, and even the format of vendor data feeds change over time. Code written for one period's conventions may silently produce wrong answers, rather than an error, on a later period with a slightly different schema.

Performance at scale. A script that works on five days of data for two assets may be far too slow on years of tick-by-tick data across thousands of instruments. Vectorized NumPy and pandas operations help, but some tasks require more specialized tools, such as columnar file formats, database systems, or distributed computing frameworks.

Environment and dependency drift. A notebook that runs correctly today may fail in a year if underlying libraries change their behavior or drop older syntax, which is why pinning dependency versions and periodically re-validating old code are standard practice in production financial systems.

Latency and real-time constraints. Some applications, particularly in trading, require code to execute within milliseconds or microseconds. Python trades raw execution speed for convenience, so performance-critical components are sometimes written in compiled languages or accelerated with tools such as Numba or Cython, while Python remains the layer for research, orchestration, and analysis.

Dependency and library version conflicts. A pandas or NumPy upgrade can silently change a function's default behavior (for example, how an aggregation handles missing values), so code not re-tested against a new library version should not be assumed to behave identically, even if it runs without error.

The gap between exploratory and production code. Code written quickly in a notebook to explore an idea is rarely suitable, unmodified, for a system that runs unattended in production. Exploratory code typically lacks the error handling, logging, and automated testing a production financial system requires, and treating the two as interchangeable is a common source of operational incidents.

2.7 Key Takeaways

Python is a practical tool for financial analysis because it supports efficient computation, flexible data handling, and reproducible workflows, and its libraries take an analyst from raw market data to descriptive analysis, forecasting, and machine learning.

The worked example showed how a handful of pandas and NumPy operations on five days of prices reveal a meaningful diversification effect, and how a formula derived on paper should match a direct computation on the data. The habits introduced here (vectorized computation, careful data cleaning, reproducible workflows) scale directly to the much larger datasets of later chapters.

Four distinctions recur, largely unchanged, in every later chapter. First, the choice between a loop and a vectorized operation is not a matter of style: it depends on whether a calculation is independent across observations (vectorize it) or sequential, like the compounding balance example (a loop, or an equivalent

Wulin.Suo@Queensu.ca

closed-form formula, is required). Second, a performance number is only as meaningful as the choices behind it: simple or log returns, adjusted or raw prices, and an annualization that assumes independent returns and, on a short sample, turns a lucky week into an implausible Sharpe ratio. Third, combining data from multiple sources, the `merge` and `groupby` operations of Section 2.3.2, is a central skill, since almost no real financial analysis works from a single tidy table. Fourth, every code example in this chapter was written to be checked, by hand, as with the four AssetA and AssetB returns, or by running the companion notebook and comparing outputs line by line. Later chapters rely on this habit of verifying rather than trusting a computation once calculations become too complex to check by hand, such as the maximum-likelihood-estimated GARCH models of Chapter 7 or the gradient-boosted trees of Chapter 6.

2.8 Suggested Exercises

2.1 Write a Python function that computes the future value of an investment given a price, an annual rate, and a number of years, and use it to verify the two examples in Section 2.1.2.

2.2 Recreate the `prices` DataFrame from Section 2.1.3, add a third asset, `AssetC`, with your own five prices, and compute its returns using `pct_change()`.

2.3 Using the covariance formula for portfolio variance, compute the volatility of a portfolio with weights 0.7 on AssetA and 0.3 on AssetB, and check your answer against a direct computation on `portfolio_returns`.

2.4 Modify the data-cleaning example to simulate two consecutive missing values in AssetB, apply forward-fill, and discuss whether this is still a reasonable choice.

2.5 Plot the cumulative growth of $1 invested in AssetA, AssetB, and the equally weighted portfolio on the same chart, and describe what the chart shows about diversification that the summary statistics alone do not.

2.6 Explain why a positive correlation between two assets would change the diversification benefit computed in Section 2.4.1, and recompute the portfolio volatility assuming a correlation of $+0.90$ instead of -0.896, holding the individual volatilities fixed.

2.7 Describe one concrete situation in which look-ahead bias could enter a Python backtesting script without the analyst noticing, and propose a coding practice that would prevent it.

2.8 Rewrite the `classify_return` function from Section 2.1.2 as a single list comprehension using a conditional expression, and verify it produces the same labels for the four AssetA returns.

2.9 Using `np.select` as in Section 2.2.2, classify AssetB's four returns into "strong gain" ($> 1\%$), "small gain" ($> 0\%$), "flat," or "loss," and compare the result to AssetA's classification.

2.10 Extend the `securities_df` and `shares` example in Section 2.3.2 with one more security of your choosing, recompute `position_value`, and recompute the sector summary.

2.11 Extend the `extended_prices` series in Section 2.3.2 with five more business days of your own prices, and resample the full series to weekly frequency using both `.last()` and `.mean()`; explain when each aggregation would be the more appropriate choice.

2.12 Using the pattern in Section 2.5.1, describe (without necessarily running the code) why a loop that appends to a Python list one element at a time is typically slower than filling a pre-allocated NumPy array by index, although neither calls a vectorized function.

2.13 Using this chapter's `Portfolio` class, instantiate it with weights 0.7 on AssetA and 0.3 on AssetB, and verify that `portfolio.volatility` matches the answer you computed by hand in Exercise 2.3.

2.14 Add a second unit test, `test_weights_sum_to_one`, that asserts a portfolio's weights sum to 1.0 within a small tolerance, and explain what kind of real coding mistake this test would catch that the variance-formula test would not.

Wulin.Suo@Queensu.ca

2.15 Using Section 2.4.4, compute AssetB's four log returns and verify that they sum to ln(52.02/50.00). Then compute the equally weighted portfolio's return on the second day three ways, as the average of the two simple returns, as the average of the two log returns, and as the log of the portfolio's gross return, and explain which two of the three must agree exactly.

2.16 For the 70/30 portfolio of Exercise 2.3, compute the daily mean, the daily volatility, the annualized volatility, and the daily and annualized Sharpe ratios (risk-free rate zero). State the two assumptions behind the annualization, and use the approximation $1/\sqrt{\text{years of data}}$ to judge how much the annualized Sharpe ratio actually tells you.

2.17 Suppose AssetB paid a \$0.50 dividend and went ex-dividend on January 5, the day its price fell from \$51.26 to \$51.00. Compute that day's price return and total return, the dividend adjustment factor, and AssetB's maximum drawdown computed from raw and from adjusted prices. What does the comparison say about drawdowns measured on unadjusted data?

2.18 **Challenge (real data).** Download daily prices for two exchange-traded funds, the SPDR S&P 500 ETF (`SPY`) and the iShares 20+ Year Treasury Bond ETF (`TLT`), from January 1, 2015 through December 31, 2024 with `yfinance` (`yf.download(["SPY", "TLT"], start="2015-01-01", end="2025-01-01")`, which returns adjusted prices). (a) Compute each fund's annualized mean return, annualized volatility, and annualized Sharpe ratio, using a constant risk-free rate of 2% a year, and each fund's maximum drawdown. (b) Compute the 63-trading-day (roughly quarterly) rolling correlation between the two funds' daily returns, plot it, and describe how it changed over the decade, in particular after 2021. (c) Repeat part (a) with `auto_adjust=False` and the raw `Close` column, and explain which fund's numbers move more, and why.

2.9 Further Reading

- McKinney (2022), *Python for Data Analysis*. Written by pandas' original author, this is the definitive reference for the DataFrame, groupby, merge, and resample operations introduced in this chapter, covering many more edge cases and options than a single chapter can.
- VanderPlas (2022), *Python Data Science Handbook*. A broader tour of NumPy, pandas, and visualization that fills in the general-purpose Python and data-science background this chapter assumes, alongside its finance-specific examples.
- Hilpisch (2018), *Python for Finance*. Extends this chapter's two-asset portfolio example toward the derivatives pricing, risk management, and algorithmic trading applications covered later in this book, using the same core pandas and NumPy toolkit.
- Hilpisch (2020), *Python for Algorithmic Trading*. A practical, code-heavy treatment of building and backtesting trading systems in Python, a natural next step after this chapter's vectorization and performance material for readers headed toward Chapters 13 and 14.

Wulin.Suo@Queensu.ca